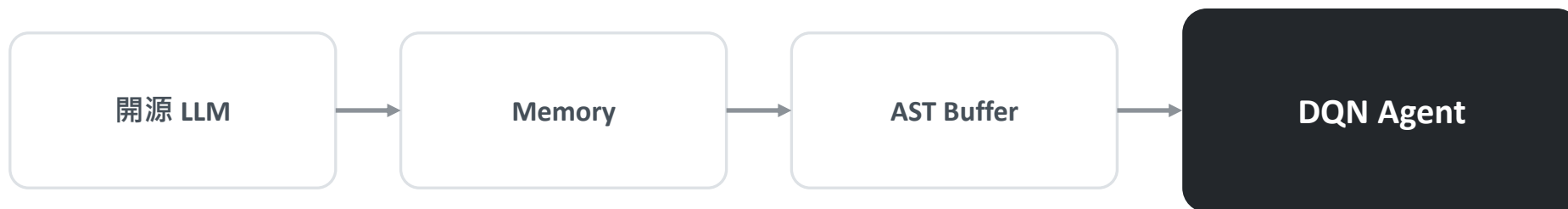


Hermes-DQN

記憶擴增之大型語言模型獎勵設計框架



陳盛茂 · 林仙安 · 辛語柔 · 陳冠宇

國立中興大學 資訊管理學研究所

Outline 報告大綱

1

Introduction

研究背景與動機

2

Related Work

相關研究探討

3

Proposed Scheme

Hermes-DQN 系統架構

4

Simulation

實驗設計與結果分析

5

Conclusion

結論與未來展望

本場聚焦

一個反直覺發現：

「記憶」對 LLM 獎勵設計
並非永遠有益——

效益取決於獎勵密度。

1

Introduction

研究背景與動機

DRL 的瓶頸與 EUREKA 的三大侷限

Motivation

獎勵設計兩難

稀疏 (Sparse) → 難以學習
人工塑形 (Dense) → 引入偏差

EUREKA (ICLR 2024)

以 GPT-4 自動撰寫獎勵函數
83% 任務超越人類專家

● 依賴商業 API

GPT-4 費用高、無法離線、不可重現

● 缺乏跨輪記憶

每次迭代從零開始，無法累積經驗

● 忽略緩衝區失效

獎勵變動使 DQN 發生災難性遺忘

→ 三大缺口同時存在，尚無單一框架完整解決。

研究契機

若能用開源模型 + 記憶 + 緩衝區管理
一次補上這三個缺口，
自動化獎勵設計才可能真正落地。

開源化

能否以輕量級開源模型取代 GPT-4？

核心問題

同時解決「開源化、記憶化、
緩衝區穩定化」之後，
LLM 設計的獎勵函數
能否普遍有效？

記憶化

跨迭代累積經驗是否「必定」帶來正向效益？

緩衝穩定

如何在獎勵非穩態下保護 DQN 歷史樣本？

預期 (假說)

記憶機制應能幫 LLM 寫出更好的獎勵，且效益跨任務一致。

→ 本研究將檢驗它。

2

Related Work

相關研究探討

	核心模型 / API	跨代記憶 (Memory)	緩衝區遺忘處理
EUREKA (Ma et al., ICLR 2024)	依賴 GPT-4	無記憶	無處理
CARD (Sun et al., 2024)	依賴 LLM 評論員	無原生環境記憶	無處理
LEARN-Opt (Cardenoso & Caarls, 2025)	開源小模型	無記憶	無處理

Research Gap | 當前三大侷限

1. 過度依賴昂貴且無法離線的商業 API
2. 缺乏跨迭代的經驗累積 (Memory)
3. 未處理獎勵變更導致的歷史樣本失效

Pillar 2 | 記憶擴增之 LLM 智能體

Related Work

「記憶架構是近期智能體進步的最大推手：OSWorld 準確率 12% → 66.3%」

— Stanford HAI 2026 AI Index

Procedural Memory

SKILL.md — 固化操作規則

Semantic Memory

USER / MEMORY.md — 成功/失敗範例知識庫

Episodic Memory

SQLite FTS5 — 完整歷史與 Fitness 指標

Working Memory

Prompt Context — 當前檢索的 Top-K 先驗

Research Gap

效益的未定之天

記憶對 Agent 有益，但對「DQN 獎勵設計」是否有絕對助益？

尤其密集 vs 稀疏環境的差異，至今缺乏嚴謹的 DRL 多任務消融實證。

Pillar 3 | 非穩態獎勵與底層穩定性 (Bellman Drift)

Related Work

LLM 變更獎勵函數



目標分佈飄移 (Ng et al., 1999)



Bellman 算子漂移 / CHAIN 效應



歷史樣本失效 (災難性遺忘)

過去解法 (Value-side)

GB-DQN (Lee & Lee, 2025)：以梯度增強處理「價值函數面」的遺忘。

Research Gap

缺乏直接針對「重播緩衝區 (Buffer-side)」的處理。

→ 需建立一套依「程式碼差異 (AST Diff)」決定樣本保留策略的底層機制，與 value-side 工作正交互補。

Synthesis | Hermes-DQN 的研究定位

Positioning

文獻缺口 (Limitations)

依賴閉源 API、無記憶累積

DRL 任務中記憶機制缺乏實證

緩衝區樣本遺忘問題未解

Hermes-DQN 架構創新

開源 LLM 替代 + 導入四層記憶閉環架構

首度發現記憶在密集環境的負向效應 ($p=0.0317$)，並以 DQN 變體驗證模型無關性

獨創 AST 感知重播緩衝管理，依結構差異執行 KEEP / PARTIAL / DECAY / CLEAR

Hermes-DQN 整合三大領域，建立開源自動化獎勵設計的研究基準。

3

Proposed Scheme

Hermes-DQN 系統架構

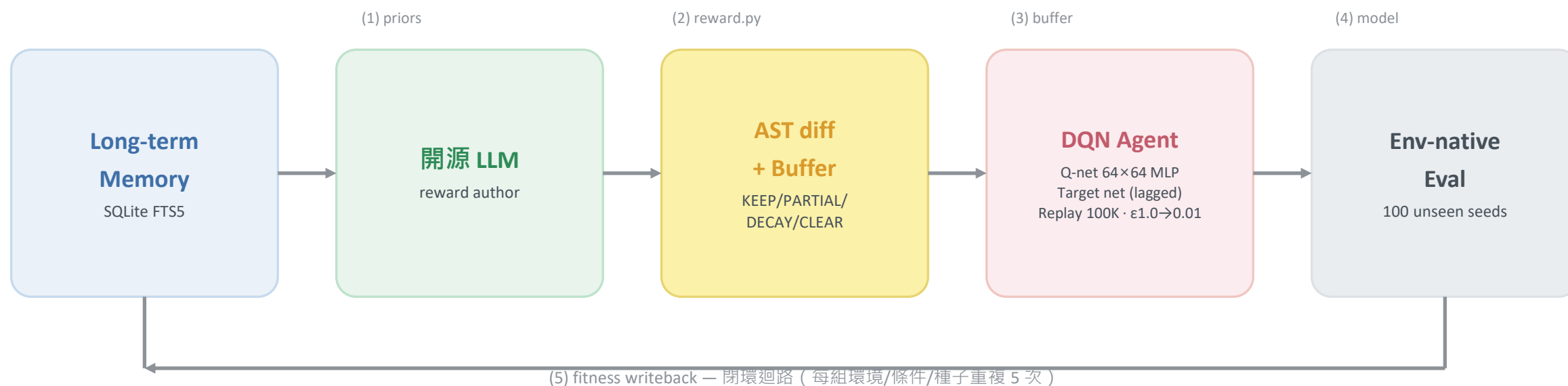
系統架構資料流 (The Big Picture)

Architecture

● 子系統一：跨世代學習能力

● 子系統二：開源且可重現的獎勵引擎

● 子系統三：抑制漂移與災難性遺忘



7 步驟迭代閉環：系統運轉引擎

Closed Loop



放大檢視 I：Hermes 四層記憶架構

Zoom-in I

Working Memory

作用中記憶

Prompt Context (本輪檢索出的 top-K 先驗)

Episodic Memory

情節記憶

SQLite FTS5 (Fitness 指標、獎勵原始碼、差異分類)

Semantic Memory

語義記憶

MEMORY.md (長期成功 / 失敗教訓)

Procedural Memory

程序記憶

SKILL.md (固化操作規則，如要求純函數)

檢索漏斗

Retrieval Funnel

查詢鍵 =
環境名稱 + 上一輪
Fitness 指標
→ 取 Top-K

放大檢視 II：開源獎勵生成器

Zoom-in II

輸入 (Input)

Task Spec (環境描述、觀察空間)

歷史先驗 (來自四層記憶)

Fitness 回饋

引擎：開源 LLM

Stateless 無狀態：
所有跨迭代訊息
皆由記憶區塊載入

輸出 (Output)

```
def reward_fn(  
    obs, action,  
    next_obs, done  
) -> float
```

安全沙箱 (Security Sandbox)

語法 AST 解析 + 沙箱執行驗證 (L2 subprocess isolation)：僅允許決定性 Python，禁止網路與檔案 I/O。

放大檢視 III：AST 感知 Replay Buffer 策略

Zoom-in III

相似度決策光譜 (Similarity Decision Spectrum)



差異越大，清除越狠

Linear degradation of prior experience based on structural drift

AST 差異如何計算？（補充）

How it works

```
# 兩段原始碼字串的純函數
old = ast.parse(prev_src)
new = ast.parse(curr_src)
kind = classify(old, new)
policy = DECIDE[kind]
apply(prev_buffer, policy)
```

以 Python 內建 ast 模組分別解析新舊獎勵原始碼，比較語法樹結構；分類器與策略查表皆為「兩個字串的決定性純函數」，可重現、無副作用。

四類結構差異 → 對應策略

IDENTICAL

語法樹完全相同

→ KEEP

SIGNATURE_ONLY

簽章不變、函式體變動

→ PARTIAL_KEEP

STRUCTURAL_DIFF

控制流變動、多數運算元延續

→ DECAY

TOTAL_REWRITE

結構幾無重疊

→ CLEAR

智能體變體：展現系統的模型無關性

Model-Agnostic

Hermes 框架與底層 DQN 架構完全解耦，可透過 `--dqn-variant` 無縫切換。

1 Vanilla

```
use_double=False  
dueling=False
```

標準 DQN

2 Double DQN

```
use_double=True  
dueling=False
```

分離線上/目標網路，消除最大化偏誤

3 Dueling DQN

```
use_double=False  
dueling=True
```

Q 分解為 $V(s)$ 與優勢 $A(s,a)$

4 Double + Dueling

```
use_double=True  
dueling=True
```

兩者結合

嚴謹驗證：六種消融實驗條件 (Ablation Matrix)

Ablation

條件 (Condition)	獎勵來源	記憶 (FTS5)	AST 緩衝	迭代
B0-env-native	原生獎勵 Baseline 基準線	—	—	1
B1-handcrafted	人工塑形 人類專家基準	—	—	1
B2-gemma-oneshot	LLM 單次 LLM 基準	—	—	1
B3-no-memory	LLM 驗證記憶的獨立影響	—	✓	5
B3-no-AST	LLM 驗證緩衝管理的獨立影響	✓	—	5
B3-hermes-full	LLM 完整系統	✓	✓	5

總規模：4 環境 × 6 條件 × 5 種子
= 120 次完整訓練

4

Simulation

實驗設計與結果分析

實驗設定 (Setup)

Setup

四個古典控制環境 (涵蓋獎勵密度兩端)

環境	密度	觀測	動作	解題門檻
LunarLander-v3	密集	8	4	mean $\geq$ 200
CartPole-v1	稀疏	4	2	mean $\geq$ 475
MountainCar-v0	稀疏	2	3	mean $\geq$ -110
Acrobot-v1	稀疏	6	3	mean $\geq$ -100

訓練配置

硬體 RTX 4090 $\times$ 1 · Win11 · CUDA 12.1

環境 Python 3.11 · PyTorch 2.5.1

DQN 64 $\times$ 64 MLP · lr=5e-4 · γ =0.99

batch=64 · replay 100K

ϵ 於 50K 步線性衰減 $\rightarrow$ 0.01

目標網路每 1000 步更新

訓練種子 42, 43, 44, 45, 46

評估種子 10000–10099 (與訓練互斥)

總訓練 120 次 = 4 $\times$ 6 $\times$ 5

主要指標：env_native_mean

以「環境原生（未經塑形）獎勵」在 100 個未見種子 (10000–10099) 上的平均回報——這是跨條件唯一公平的衡量基準。

檢定方法

雙尾 Mann-Whitney U
($\alpha = 0.05$ ，非參數、適合 $n=5$)

信賴區間

Bootstrap 5000 次再抽樣
95% 信賴水準

「勝出」三條件

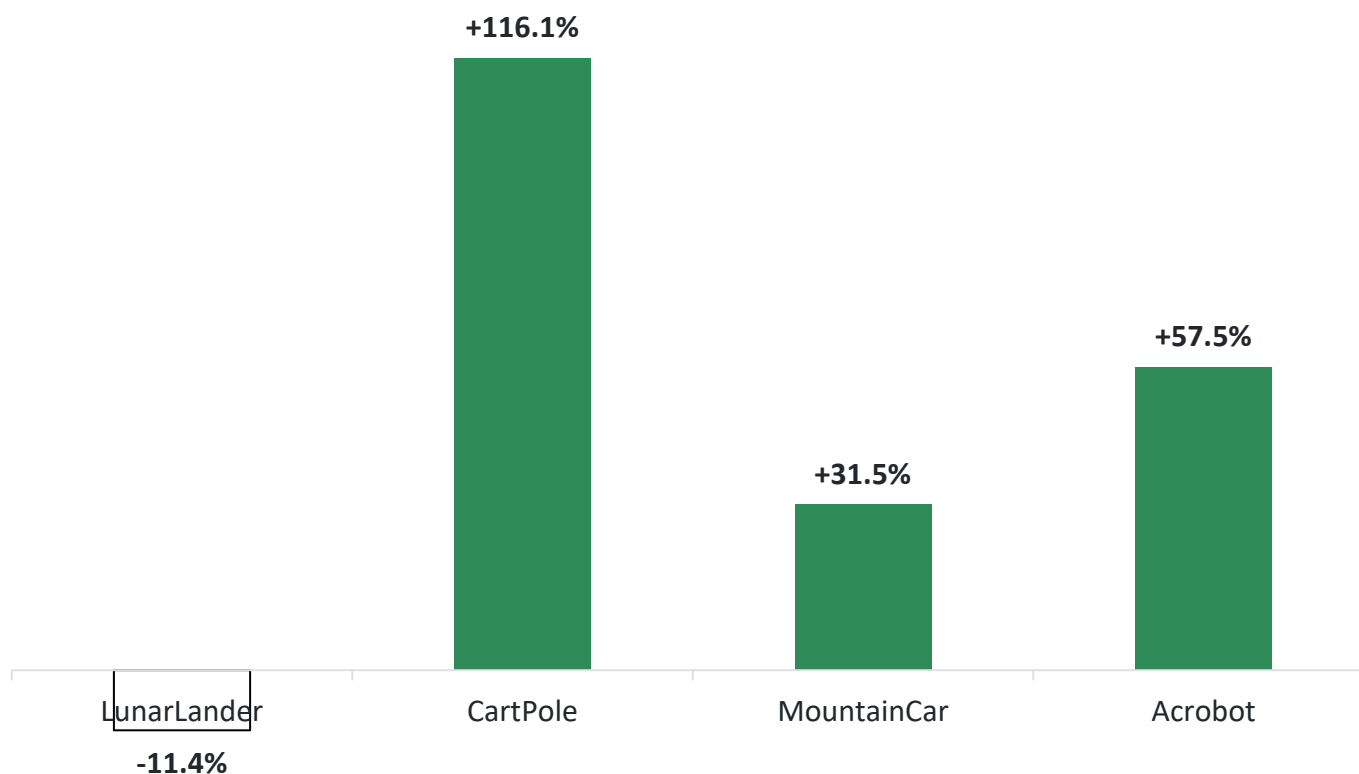
$p < 0.05$ 、 $|\Delta|/|\text{base}| \geq 10\%$ 、
信賴區間不重疊

統計力限制（誠實揭露）

$n=5$ 僅對「大效應」(Cohen's $d \geq 1$) 具穩定偵測力；中等效應在此樣本下難以定論。種子全數保留、不剔除崩潰種子（如 Acrobot 之 B0），故其變異與信賴區間被膨脹。

Part 1 結果：稀疏「大勝」、密集「反轉」

Part 1 · Results



B3-hermes-full vs B0-env-native (p : 1.00 / 0.0317 / 0.0112 / 0.0952)

稀疏環境：顯著正向

CartPole +116% (p=0.0317)

MountainCar +31.5% (p=0.0112)

→ 原生訊號微弱時，LLM 解鎖學習瓶頸。

密集環境：記憶帶來傷害

LunarLander：相對「無記憶版」

衰退 -38.3% (p=0.0317)

→ 原生獎勵已富梯度時，跨迭代記憶反而累積衝突 (Additive Shaping Conflict)。

解鎖極限：稀疏環境下的獎勵塑形

Sparse Win

CartPole-v1 (存活挑戰)

原生 DQN (B0)

154.8



Hermes-full (B3)

334.4

效能翻倍，解鎖原生無法解的任務

+116%

MountainCar-v0 (動量挑戰)

原生 DQN (B0)

-193.4



Hermes-full (B3)

-132.5

五個種子一致收斂 (std=3.08)

+31.5%

為什麼稀疏環境會大勝？

原生獎勵幾乎不含學習訊號

(僅二元存活 / 固定時間懲罰) · 原生 DQN 在預算內幾乎解不出題。

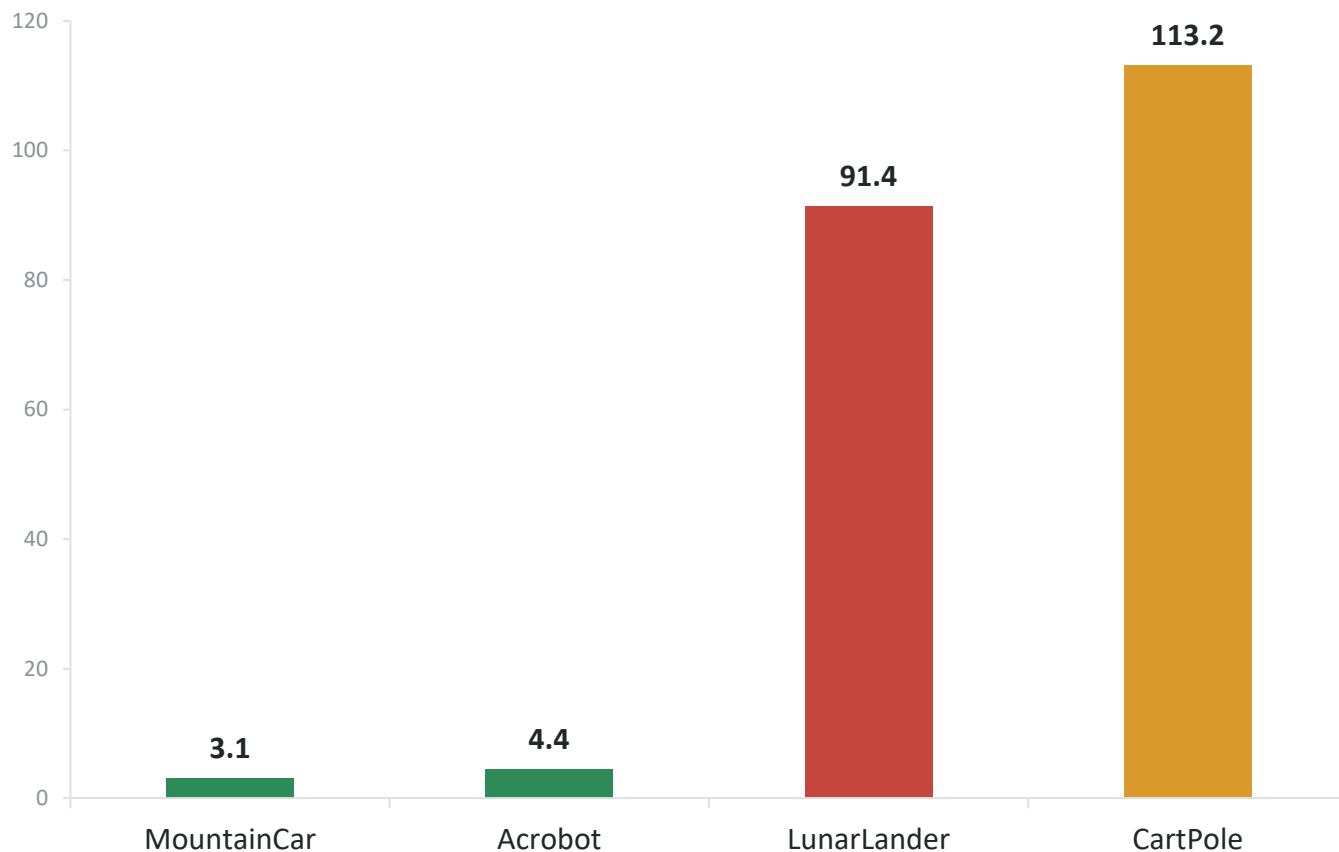
LLM 寫出的塑形 = 唯一明確子目標

MountainCar 上 Gemma 寫出「每步 +0.5 動量」，媲美經典文獻的人類專家塑形。

B0 成功率：CartPole 0% · MountainCar 0% · Acrobot 62%

病理分析：變異性指紋 (Variance Signature)

Variance



B3-hermes-full 各環境逐種子標準差 (越低越穩定)

簡單物理 / 稀疏 → 極端穩定

MountainCar (3.08)、Acrobot (4.39) 緊湊收斂——單一明確目標
· LLM 穩定收斂至近最佳唯一解。

豐富塑形空間 → 高變異風險

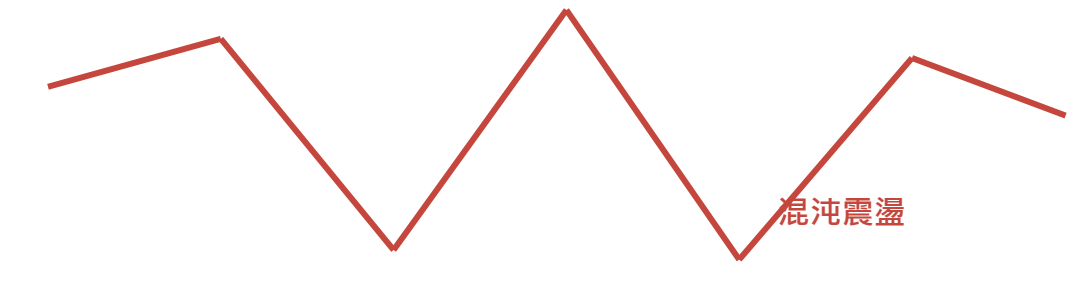
LunarLander (91.40) 高低落差極大。

致命的 seed_43 (得分 11.6) : LLM 生成「重罰垂直速度」+
「獎勵腳部接觸」的矛盾組合，

智能體學會「貼地懸停但不降落」的退化策略。

軌跡透視：記憶機制如何引發干擾？

LunarLander-v3 (密集獎勵)



MountainCar-v0 (稀疏獎勵)



關鍵詮釋

當原生獎勵已有強烈梯度時，
跨迭代記憶反而讓 LLM
累積了「衝突的干擾項」。

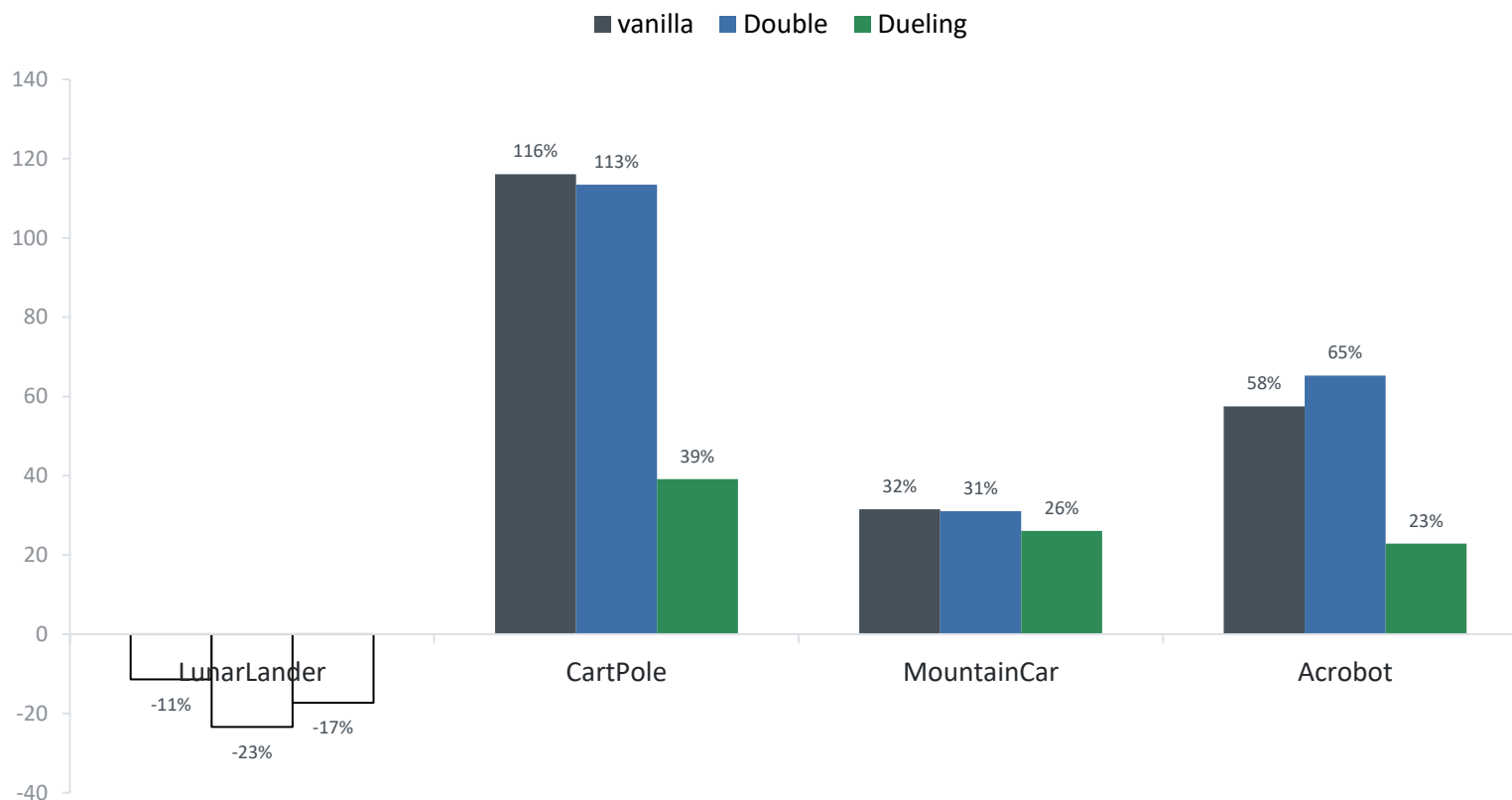
Additive Shaping Conflict

(示意圖：依論文圖 5 之軌跡型態繪製)

Part 2 泛化驗證：這只是底層演算法的特例嗎？

Part 2

固定獎勵管線，僅切換 DQN 變體 (vanilla / Double / Dueling) —— $\Delta\%$ = Hermes vs B0。



三項確認

- ① 稀疏勝、密集反轉的型態
在三種變體下完整複現。
- ② Hermes 在變體間無顯著差異
(all $p > 0.3$)。
- ③ 證實模型無關性：獎勵設計
的作用獨立於價值網路架構。

完整結果數據表 (附錄)

Appendix · Table 1

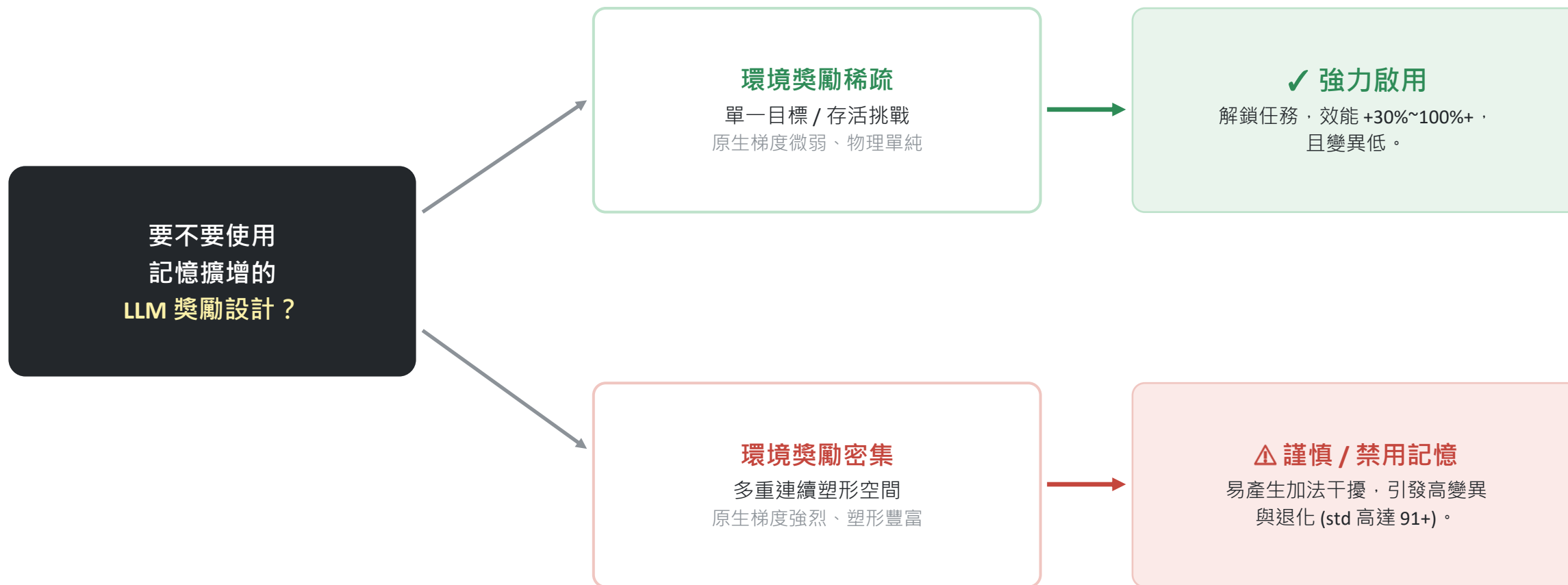
env_native_mean (n=5) | 粗綠 = 相對 B0 顯著勝出，紅斜 = 相對 no-memory 顯著敗退

條件	LunarLander (密集)	CartPole (稀疏)	MountainCar (稀疏)	Acrobot (稀疏)
B0-env-native	173.22	154.80	-193.44	-194.96
B1-handcrafted	77.77	160.19	-140.40	-185.28
B2-gemma-oneshot	152.65	187.64	-153.09	-83.21
B3-hermes-full	153.56	334.44	-132.53	-82.92
B3-no-memory	248.77	243.21	-168.55	-83.23
B3-no-AST	95.42	220.81	-134.59	-83.58

一句話：三個稀疏環境 B3-hermes-full 相對 B0 +31.5%~+116%；唯一的密集環境 LunarLander 反而被「記憶」拖累 -38.3%。

結論法則：獎勵密度假說 (Reward Density Hypothesis)

Decision Rule



5

Conclusion

結論與未來展望

Hermes-DQN：貢獻與核心發現

Conclusion

系統性整合「開源 LLM + 四層記憶 + AST 感知緩衝」，歷經 4 大環境、120 次嚴格訓練與消融實驗。



核心發現：破除「記憶必定有益」的迷思

記憶對 LLM 獎勵設計的效益取決於「獎勵密度」——稀疏環境大幅受益，密集環境反而受害。

這是該領域首次提出、且具統計顯著的「任務相依性反轉」報告。

兩個世界：稀疏 vs 密集

Two Regimes



稀疏獎勵 Sparse Environments

任務 CartPole · MountainCar · Acrobot

效能 +25% ~ +116% 顯著提升 (vs B0)

特性 正向協同、變異極低 (std 3~4)

結論 開源 LLM + 記憶發揮強大優勢



密集獎勵 Dense Environments

任務 LunarLander (唯一密集環境)

效能 記憶導致效能衰退 -38%

特性 高變異 (std 91+)、可能策略崩潰

結論 該領域首次具統計顯著的負向報告

研究限制 (Constraints & Limitations)

Limitations

小樣本規模

n=5 僅對大效應 ($d \geq 1$) 有力；部分記憶效應方向明確卻無法定論。後續以 n=10/20 重做。

密集環境僅一個

「密度」與「塑形空間」在四環境面板中部分混淆；負向發現目前僅由 LunarLander 支撐，需 LunarLanderContinuous 分離兩變數。

B1 為暫時基準

人工塑形由作者自撰，非第三方；不足以作為決定性人類基準。

單一 LLM

變異可能與該模型抽樣特性有關；Llama 3.3 / Qwen 3 / DeepSeek-V3 重做為自然延伸。

固定 5 輪迭代

閉環迴圈硬編碼為 5 輪；更長迭代能否逆轉負向效應未知。

僅部分 Rainbow

只實作 Double + Dueling；PER / Multi-step / Noisy / 分佈式 Q 留待後續。

Step 1 連續控制擴張

進軍 MuJoCo 等高維連續動作基準。

Step 2 勢能約束生成

將理論邊界注入 LLM Prompt，從數學上保證生成的塑形獎勵不破壞原始最優策略。

Step 3 自適應迭代機制

捨棄固定輪數，改以 Fitness 曲線動態判定是否終止迭代迴圈。

Step 4 多模型基準整合

引入多種開源模型橫向 Benchmark，隔離出模型特定的偏誤特徵。